

Troubleshooting guide

Reference for common issues and resolutions across the Helix workflow.

1. Pre-Validation Failure Reference

If you encounter pre-validation failures, use the sections below to diagnose and resolve them.

1.1 Docker Build Fails

- Common cause: missing git configuration in the Dockerfile.
- Add the following line to your `Dockerfile.helix` to resolve authentication issues:

```
RUN git config --unset-all http.https://github.com/.extraheader  
|| true
```

[Copy](#)

- Place this line before any git operations in your Dockerfile. After editing, push to the `docker-golden-solution` branch, merge it, and rebase to continue your work.
- Ensure the `.github/workflows` folder contains the `build-push-gar.yml` file (see Section 3 below). Without it, you will encounter a "manifest unknown" error.

Example Dockerfile.helix



1.2 Docker Build Fails — "No space left on device"

The system-generated `Dockerfile.helix` can be heavy: it pulls a large base image and installs many packages, which can exhaust the runner's disk and fail the build with an error like:

```
No space left on device
```

[Copy](#)

Fix: replace the `Dockerfile.helix` with a lighter base image, then merge to `main`.

1. Switch the `FROM` line to a slim variant of the same language (e.g. `python:3.11-slim` instead of `python:3.11`, `node:20-slim` instead of `node:20`).
2. Remove any `apt-get` or `apk` installs that aren't strictly required by the test suite. Keep `git` and `ca-certificates` — those are mandatory.

3. Don't COPY the entire repo if only a subset is needed for tests; copy what the test runner actually needs.
4. Test the build locally before pushing: `docker build -f .helix/Dockerfile.helix -t helix-tests .`

Once it builds locally, open a PR on the `docker-golden-solution` branch to merge the lighter Dockerfile to `main`. After it merges, rebase your `golden-solution` branch and continue.

Lighter images are the default fix

Most 'No space left' failures resolve by switching from a full base image to its slim variant. Only drop additional packages if slim alone isn't enough.

1.3 P2P Failed on Base

P2P is designed to pass on base. If it fails:

- Ensure there are no live tests included.
- Skip tests unrelated to your task.
- Review the runscript for missing dependencies if the issue persists.

1.4 Git Application Failed

- Ensure your commits are based on the current main branch.
- Confirm all git-related steps are included and correctly ordered in the runscript.
- Ensure that your changes on each commit do not conflict with each other (make updates to the same files).

1.5 F2P Failed on Base

Your F2P test cases are passing on the base code, which is not the expected behavior.

- Ensure there is a real bug being fixed by your solution.
- Add edge cases to your F2P tests so they correctly fail on the base code.

1.6 F2P Failed on Golden Solution

Your golden solution does not fully resolve the issue.

- Review your golden solution to ensure it completely addresses the bug.
- Add additional edge case tests alongside existing ones from the existing-solution branch to improve coverage.

2. Understanding Pass@

You do not need to worry about Pass@ during development. Here is how it works:

1

Approval

2

Automated build

3

Stumping the model

4

Trajectory generated

Until a failure is generated, focus on:

- Building a strong, complete solution.
- Fixing any remaining issues.
- Ensuring quality and completeness of your implementation.
- Writing stronger, more comprehensive F2P tests to increase the chances of stumping the model.

3. Required GitHub Workflow Files

Your main branch must contain a `.github/workflows` folder with exactly these three files and exactly the content shown below.

File 1: build-push-gar.yml

```
name: Build and Push to GAR
```

 Copy

```
on:
  push:
    branches: [master, main]

permissions:
  contents: read
  id-token: write

jobs:
  build-and-push:
    uses: handshake-coding/handshake-orchestration/.github/workflows/build-push-gar.yml@main
```

File 2: golden-solution-validation.yml

```
name: Golden Solution Validation
```

 Copy

```
on:
  pull_request:
  workflow_dispatch:

permissions:
  contents: read
  id-token: write

jobs:
  prevalidate:
    if: github.head_ref == 'golden-solution' || github.event_name == 'workflow_dispatch'
    uses: handshake-coding/handshake-orchestration/.github/workflows/golden-solution-validation.yml@main
    secrets: inherit
```

File 3: helix-validation.yml

```
name: Helix Validation
```

 Copy

```
on:
  pull_request:
  workflow_dispatch:

jobs:
  validate:
    if: github.event_name == 'workflow_dispatch' || (github.base_ref == github.event.repository.default_branch
```

```

88
    github.head_ref == 'docker-golden-solution')
    uses: handshake-coding/handshake-
orchestration/.github/workflows/helix-validation.yml@main
    secrets: inherit

```

Exact match required

There must be exactly these three files in the `.github/workflows` directory. File names must match exactly and content must be identical to the above.

4. Pull Request Requirements

4.1 Docker & Runscript Changes (docker-golden-solution branch)

For PRs that contain Docker and runscript changes:

- The PR must be created strictly from the `docker-golden-solution` branch. PRs from any other branch will not trigger the required validations.
- Two checks are required: **Helix Validation / validate / Validate .helix required files** and **Helix Validation / validate / Build & run pass@ compatible tests**.

Expected result when all checks pass:

All checks have passed

1 skipped and 2 successful checks

		Golden Solution Validation / prevalidate (pull_request) ...	Details
		Helix Validation / validate / Validate .helix required file...	Details
		Helix Validation / validate / Build & run pass@ compati...	Details

All checks have passed (1 skipped, 2 successful) — Golden Solution Validation skipped, both Helix Validation checks green.

4.2 Golden Solution Changes (golden-solution branch)

For PRs containing Golden Solution changes:

- The PR must be created strictly from the `golden-solution` branch.
- Commits and the content of commits must follow HELIX requirements.
- Only one check is required: **Golden Solution Validation / prevalidate**.

Expected result when validation is pending:



Some checks haven't completed yet

1 skipped, 2 expected, 1 successful checks

2 pending checks ▾



validate / Build & run pass@ compatible tests

Expected — Waiting for status to be reported

Required



validate / Validate .helix required files

Expected — Waiting for status to be reported

Required

1 skipped check ▾



Helix Validation / validate (pull_request)

Skipped 3 hours ago

...

1 successful check ▾



Golden Solution Validation / prevalidate / Validate golden solution via swe-runner (pull_request)

Successfu...

...

Merge pull request ▾

You can also merge this with the command line. [View command line instructions.](#)

1 skipped, 2 expected, 1 successful — Golden Solution Validation successful, Helix Validation checks pending (waiting for status).

Helix Validation stuck at 'Waiting for status'

This is not an error and not a blocker. Only the Golden Solution Validation / prevalidate check is required for golden-solution PRs.

5. Docker Seeding Workflow

Some repos do not have Docker infrastructure yet. If your task is flagged as a Docker Seeding Task (the repo has no `.helix` folder), complete this workflow first, then return to the Core Workflow at Step 4.

5.1 What You Need to Create

Add a `.helix` folder at the repo root containing three files:

- **Dockerfile.helix** — Pins the repo's language and dependencies at the current commit.
- **run-tests-eval.sh** — Runs the test suite in two modes: all tests, or specific files.
- **metadata.json** — Task metadata file.

5.2 Dockerfile.helix Requirements

- Use a base image matching the repo's language and version at the current commit.
- Install all dependencies needed to run the test suite (e.g. `pytest`, `jest`, `vitest`).
- Always include `git` and `ca-certificates` in your install step. The CI workflow runs `git` inside the container — omitting these causes "git: command not found" and SSL certificate errors. Use the package manager that matches your base image (`apt-get` for Debian/Ubuntu, `yum` or `dnf` for Amazon Linux/RHEL, `apk` for Alpine).
- Add the `git config unset` line to prevent HTTPS auth issues inside the container.

- Copy the repo contents into the image.

5.3 run-tests-eval.sh Requirements

- **No arguments** — run ALL tests in the repo (default mode for full validation).
- **With arguments** — accept comma-separated file paths and run only those tests (used by the evaluation harness).
- Call the test runner directly (e.g. pytest, jest). Do not invoke Docker inside the script — the CI workflow handles Docker.
- The script must be executable. Commit it with permission mode 100755 :

```
chmod +x .helix/run-tests-eval.sh
```

 Copy

Do not skip tests

Do not skip any tests from the existing solution branch. Never edit anything in the existing-solution branch. If the repo has many flaky tests, you may prune them so that every test that runs also passes.

5.4 Common Errors & Fixes

Error 1: Permission Denied (exit code 126)



Error 2: git Not Found (exit code 127)



5.5 Opening the PR

1

Create the branch

2

Open a PR

3

Wait for validation

4

Fix failures if needed

5

Merge and submit

6. Submit Issues

6.1 Submit button is disabled after checks pass

1

Re-run the workflow

2

Wait and refresh

3

Check for stale runs

4

Escalate if needed

6.2 Problem statement mismatch error

If you receive: "The problem_statement you pasted into the form does not match the one in .helix/metadata.json on the golden-solution branch" — and you are certain the text is identical:

- Copy both versions into an online diff checker (e.g. [diffchecker.com](https://www.diffchecker.com)) to confirm they are truly identical.
- Check for hidden whitespace, trailing line breaks, or encoding differences.
- If confirmed identical and the error persists, post your Task ID in the issues thread. This is a known platform-side false positive requiring admin resolution.

6.3 "Needs Fixing" with no feedback visible

If your task is returned with a "Needs Fixing" status but no reviewer feedback appears:

- Check whether the task is in read-only mode. If it is, no action is needed — this is expected. Wait for feedback to appear before making any changes.

- If the task is not in read-only mode and the submit button says "Address feedback under Needs Attention" but nothing is visible there, this is a platform bug.
- Post your Task ID in the review bugs thread. An admin will need to resolve it on the backend.

Do not resubmit

Do not resubmit or make changes until visible feedback is available.

7. Invalid Tasks

7.1 How to submit an invalid task

1

Fill all fields

2

Submit the task

3

Post in the thread

4

Hold the task

Active task limit

Do not claim additional tasks beyond your allowed active task limit while holding invalid ones.

7.2 Submit button is disabled on an invalid task

If the submit button is greyed out because GitHub Actions checks have not passed:

- Do not attempt to fix or pass the checks.
- Post the Task ID in the invalid tasks thread and wait for admin resolution.
- It is safe to hold the task while waiting.

8. General Notes & Reminders

- Do not make any changes to main except for Docker and runscrip files. If other changes were made, revert them and restore main to its original state.
- Do not tag individuals directly in Helix screening for Helix-related questions. Use the designated group channels.
- Most common issues have been discussed in previous threads. Check those threads before escalating.
- For onboarding and environment setup, refer to the Environment Setup section of this guide.

If solution or existing-solution code accidentally merged into main

If you find that solution code or files from the existing-solution branch were accidentally merged into main — either during your work or by a previous fellow — you can recover via the `docker-golden-solution` branch.

1. Switch to (or create) the `docker-golden-solution` branch.
2. Remove the unintended files in a new commit on that branch.
3. Open a PR from `docker-golden-solution` to main and let the validation checks run.
4. After checks pass, merge it. The diff between main and your `golden-solution` base should now only contain `.helix` changes.
5. Rebase your `golden-solution` branch on top of the updated main and continue your work.

Fixing incorrectly merged changes

If you claim a task and find that a previous fellow has incorrectly merged changes into main (e.g. a golden solution commit, extra workflow files, or a large unrelated diff), fix it:

On your `docker-golden-solution` branch, create a new commit that removes the incorrectly merged files

Open a PR from `docker-golden-solution` into main and merge it

Rebase your `golden-solution` branch on top of the updated main and continue your work

[Previous](#)[Next](#)